

Unit -1

Difference between OOP and POP

OOP:

Related with the real life objects and their properties. OOP Concepts:

1. Class and Objects
2. Data abstraction
3. Encapsulation
4. Polymorphism
5. Inheritance

POP:

Related with the conventional style. This approach is also known as the top-down approach. In this approach, a program is divided into functions that perform specific tasks. This approach is mainly used for medium-sized applications. Data is global, and all the functions can access global data. The basic drawback of the procedural programming approach is that data is not secured because data is global and can be accessed by any function. Program control flow is achieved through function calls and go to statements.

Difference between OOP and POP:

OOP	POP
Object oriented.	Structure oriented.
Program is divided into objects.	Program is divided into functions.
Bottom-up approach.	Top-down approach.
Inheritance property is used.	Inheritance is not allowed.
It uses access specifier.	It doesn't use access specifier.
Encapsulation is used to hide the data.	No data hiding.
Concept of virtual function.	No virtual function.

OOP

C++, Java.

POP

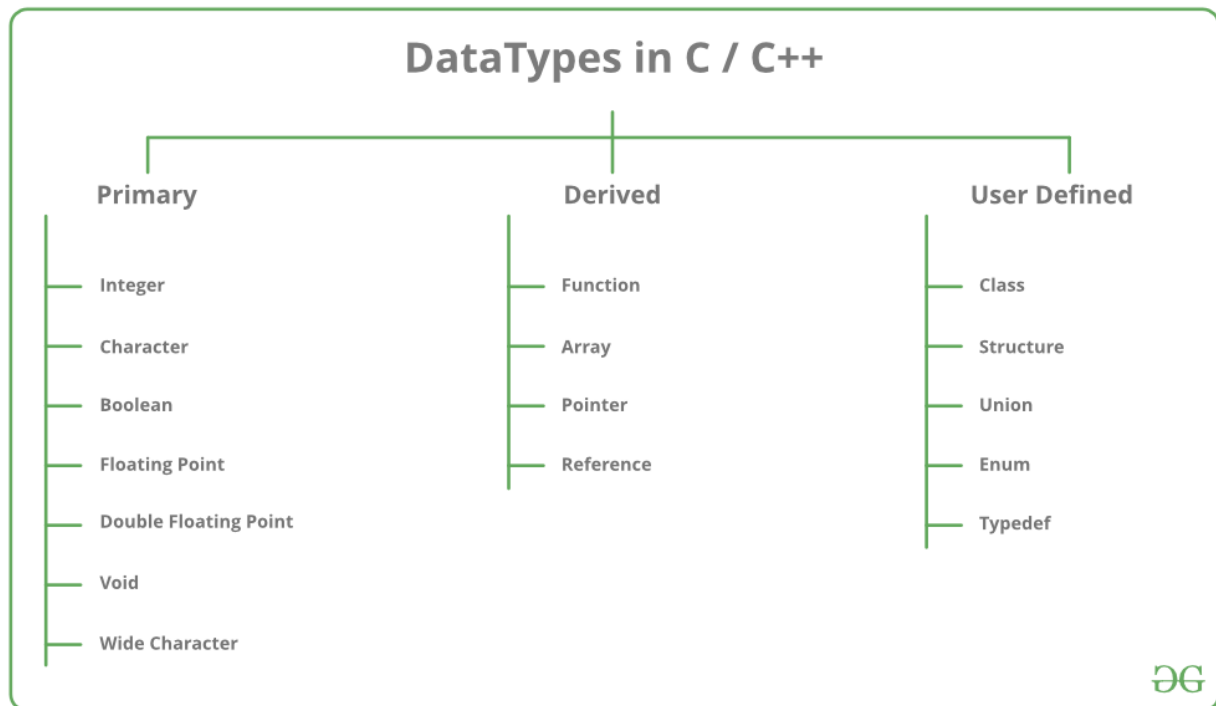
C, Pascal.

Standard Header Files And Their Uses:

1. **#include<stdio.h>**: It is used to perform input and output operations using functions **scanf()** and **printf()**.
2. **#include<iostream>**: It is used as a stream of Input and Output using **cin** and **cout**.
3. **#include<string.h>**: It is used to perform various functionalities related to string manipulation like [strlen\(\)](#), [strcmp\(\)](#), [strcpy\(\)](#), **size()**, etc.
4. **#include<math.h>**: It is used to perform mathematical operations like [sqrt\(\)](#), [log2\(\)](#), [pow\(\)](#), etc.
5. **#include<iomanip.h>**: It is used to access **set()** and **setprecision()** function to limit the decimal places in variables.
6. **#include<signal.h>**: It is used to perform signal handling functions like **signal()** and **raise()**.
7. **#include<stdarg.h>**: It is used to perform standard argument functions like **va_start()** and **va_arg()**. It is also used to indicate start of the variable-length argument list and to fetch the arguments from the variable-length argument list in the program respectively.
8. **#include<errno.h>**: It is used to perform [error handling](#) operations like **errno()**, **strerror()**, **perror()**, etc.
9. **#include<fstream.h>**: It is used to control the data to read from a file as an input and data to write into the file as an output.
10. **#include<time.h>**: It is used to perform functions related to **date()** and **time()** like [setdate\(\)](#) and [getdate\(\)](#). It is also used to modify the system date and get the CPU time respectively.
11. **#include<float.h>**: It contains a set of various platform-dependent constants related to floating point values. These constants are proposed by ANSI C. They allow making programs more portable. Some examples of constants included in this header file are- **e**(exponent), **b**(base/radix), etc.
12. **#include<limits.h>**: It determines various properties of the various variable types. The macros defined in this header, limits the values of various variable types like **char**, **int**, and **long**. These limits specify that a variable cannot store any value beyond these limits, for example an unsigned character can store up to a maximum value of **255**.

C++ Data Types

All [variables](#) use data-type during declaration to restrict the type of data to be stored. Therefore, we can say that data types are used to tell the variables the type of data it can store. Whenever a variable is defined in C++, the compiler allocates some memory for that variable based on the data-type with which it is declared. Every data type requires a different amount of memory.



Data types in C++ is mainly divided into three types:

1. **Primitive Data Types:** These data types are built-in or predefined data types and can be used directly by the user to declare variables. example: int, char , float, bool etc. Primitive data types available in C++ are:
 - Integer
 - Character
 - Boolean
 - Floating Point
 - Double Floating Point
 - Valueless or Void
 - Wide Character
2. **[Derived Data Types:](#)** The data-types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types. These can be of four types namely:
 - Function
 - Array

- Pointer
 - Reference
3. **Abstract or User-Defined Data Types**: These data types are defined by user itself. Like, defining a class in C++ or a structure. C++ provides the following user-defined datatypes:
- Class
 - Structure
 - Union
 - Enumeration
 - Typedef defined DataType

This article discusses **primitive data types** available in C++.

- **Integer**: Keyword used for integer data types is **int**. Integers typically requires 4 bytes of memory space and ranges from -2147483648 to 2147483647.
- **Character**: Character data type is used for storing characters. Keyword used for character data type is **char**. Characters typically requires 1 byte of memory space and ranges from -128 to 127 or 0 to 255.
- **Boolean**: Boolean data type is used for storing boolean or logical values. A boolean variable can store either *true* or *false*. Keyword used for boolean data type is **bool**.
- **Floating Point**: Floating Point data type is used for storing single precision floating point values or decimal values. Keyword used for floating point data type is **float**. Float variables typically requires 4 byte of memory space.
- **Double Floating Point**: Double Floating Point data type is used for storing double precision floating point values or decimal values. Keyword used for double floating point data type is **double**. Double variables typically requires 8 byte of memory space.
- **void**: Void means without any value. void datatype represents a valueless entity. Void data type is used for those function which does not returns a value.
- **Wide Character**: Wide character data type is also a character data type but this data type has size greater than the normal 8-bit datatype. Represented by **wchar_t**. It is generally 2 or 4 bytes long.

Concepts of String:

C++ provides following two types of string representations –

- The C-style character string.
- The string class type introduced with Standard C++.

The C-Style Character String

The C-style character string originated within the C language and continues to be supported within C++. This string is actually a one-dimensional array of characters which is terminated by a **null** character '\0'. Thus a null-terminated string contains the characters that comprise the string followed by a **null**.

The following declaration and initialization create a string consisting of the word "Hello". To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word "Hello."

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

If you follow the rule of array initialization, then you can write the above statement as follows –

```
char greeting[] = "Hello";
```

Following is the memory presentation of above defined string in C/C++ –

Index	0	1	2	3	4	5
Variable	H	e	l	l	o	\0
Address	0x23451	0x23452	0x23453	0x23454	0x23455	0x23456

Actually, you do not place the null character at the end of a string constant. The C++ compiler automatically places the '\0' at the end of the string when it initializes the array. Let us try to print above-mentioned string –

```
#include <iostream>

using namespace std;

int main () {

    char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};

    cout << "Greeting message: ";
```

```
cout << greeting << endl;  
  
return 0;  
}
```

When the above code is compiled and executed, it produces the following result –

Greeting message: Hello

C++ supports a wide range of functions that manipulate null-terminated strings –

Sr.No	Function & Purpose
1	strcpy(s1, s2); Copies string s2 into string s1.
2	strcat(s1, s2); Concatenates string s2 onto the end of string s1.
3	strlen(s1); Returns the length of string s1.
4	strcmp(s1, s2); Returns 0 if s1 and s2 are the same; less than 0 if s1<s2; greater than 0 if s1>s2.
5	strchr(s1, ch); Returns a pointer to the first occurrence of character ch in string s1.
6	strstr(s1, s2); Returns a pointer to the first occurrence of string s2 in string s1.

Following example makes use of few of the above-mentioned functions –

```
#include <iostream>  
#include <cstring>  
  
using namespace std;  
  
int main () {  
  
    char str1[10] = "Hello";  
    char str2[10] = "World";  

```

```

char str3[10];
int len ;

// copy str1 into str3
strcpy( str3, str1);
cout << "strcpy( str3, str1) : " << str3 << endl;

// concatenates str1 and str2
strcat( str1, str2);
cout << "strcat( str1, str2): " << str1 << endl;

// total length of str1 after concatenation
len = strlen(str1);
cout << "strlen(str1) : " << len << endl;

return 0;
}

```

When the above code is compiled and executed, it produces result something as follows –

```

strcpy( str3, str1) : Hello
strcat( str1, str2): HelloWorld
strlen(str1) : 10

```

The String Class in C++

The standard C++ library provides a **string** class type that supports all the operations mentioned above, additionally much more functionality. Let us check the following example –

```

#include <iostream>
#include <string>

using namespace std;

int main () {

    string str1 = "Hello";
    string str2 = "World";
    string str3;
    int len ;

    // copy str1 into str3
    str3 = str1;
    cout << "str3 : " << str3 << endl;

    // concatenates str1 and str2
    str3 = str1 + str2;
    cout << "str1 + str2 : " << str3 << endl;

    // total length of str3 after concatenation

```

```
len = str3.size();  
cout << "str3.size() : " << len << endl;  
  
return 0;  
}
```

When the above code is compiled and executed, it produces result something as follows –

```
str3 : Hello  
str1 + str2 : HelloWorld  
str3.size() : 10
```


C++ Classes and Objects

Class: A class in C++ is the building block, that leads to Object-Oriented programming. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A C++ class is like a blueprint for an object.

For Example: Consider the Class of **Cars**. There may be many cars with different names and brand but all of them will share some common properties like all of them will have *4 wheels, Speed Limit, Mileage range* etc. So here, Car is the class and wheels, speed limits, mileage are their properties.

- A Class is a user defined data-type which has data members and member functions.
- Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions defines the properties and behavior of the objects in a Class.
- In the above example of class *Car*, the data member will be *speed limit*, *mileage* etc and member functions can be *apply brakes*, *increase speed* etc.

An **Object** is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Defining Class and Declaring Objects

A class is defined in C++ using keyword `class` followed by the name of class. The body of class is defined inside the curly brackets and terminated by a semicolon at the end.

keyword **user-defined name**

[illegible]

Declaring Objects: When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Syntax:

ClassName **ObjectName**;

Accessing data members and member functions: The data members and member functions of class can be accessed using the dot('.') operator with the object. For example if the name of object is *obj* and you want to access the member function with the name *printName()* then you will have to write *obj.printName()* .

Accessing Data Members

The public data members are also accessed in the same way given however the private data members are not allowed to be accessed directly by the object. Accessing a data member depends solely on the access control of that data member.

This access control is given by [Access modifiers in C++](#). There are three access modifiers : **public, private and protected**.

Member Functions in Classes

There are 2 ways to define a member function:

- Inside class definition
- Outside class definition

To define a member function outside the class definition we have to use the scope resolution :: operator along with class name and function name.